

Pipeline MLOps de bout en bout — Telco Customer Churn

Projet final du cours MLOps (M1 Big Data & IA) — Fatima Amrouche

Généré le 17/09/2026 à partir des runs MLflow et des figures du dépôt github.com/fatimaamrch/PROJET_MLOps.
Modèle en production : **v3 (logreg, ROC-AUC test 0.840)**.

1. Objectif et périmètre

Construire un projet de machine learning complet et reproductible, exécutable entièrement en local, dans lequel **MLflow trace l'intégralité des expériences** (paramètres, métriques, artefacts, données, modèles) et sert de **Model Registry**, puis **déployer** le modèle retenu derrière une API. Le cas d'usage est la prédiction du *churn* (résiliation) des clients d'un opérateur télécom : classification binaire sur données tabulaires mixtes.

7,021

clients après nettoyage (sur 7,043)

19

features (3 numériques, 16 catégorielles)

26.4%

taux de churn (classe positive)

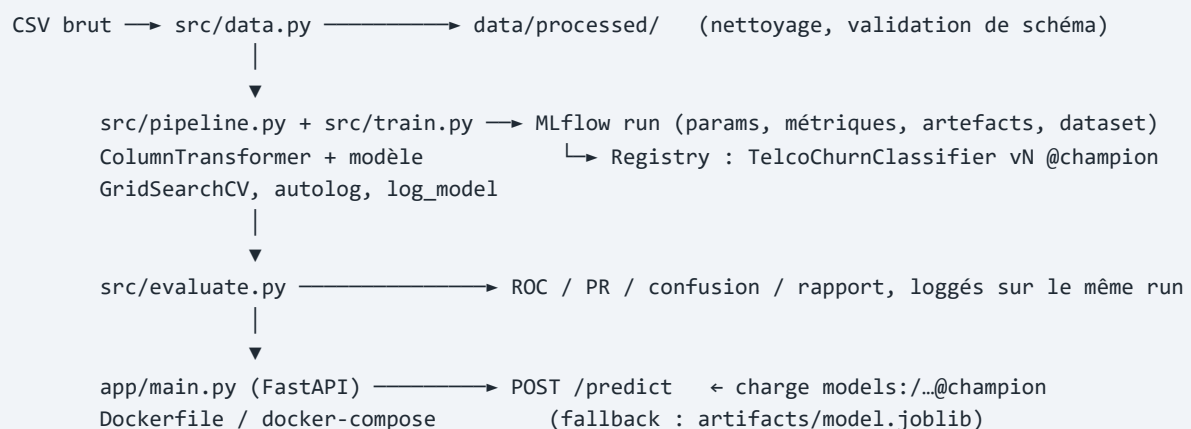
3

versions dans le Model Registry

0.840

ROC-AUC du champion (hold-out)

2. Architecture du projet

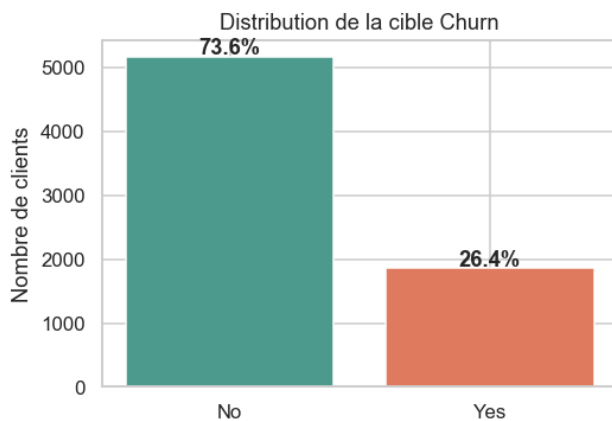


Composant	Implémentation
Configuration centralisée	configs/config.yaml : chemins, cible, features, grilles d'hyperparamètres, CV, MLflow, seuil
Données	src/data.py : téléchargement idempotent, nettoyage, validation de schéma (échec immédiat si colonne manquante)
Modélisation	src/pipeline.py : Pipeline(ColumnTransformer(imputer+scaler imputer+one-hot), modèle)
Entraînement / tracking	src/train.py : GridSearchCV (StratifiedKFold 5, ROC-AUC), autolog, dataset lineage, signature, deps épinglées, registry + alias
Évaluation	src/evaluate.py : hold-out, ROC / PR / matrice de confusion, rapport de classification, prédictions CSV, loggés sur le run d'origine
Déploiement	app/main.py (FastAPI + validation Pydantic), Dockerfile, docker-compose.yml (serveur MLflow + API)
Qualité / automatisé	22 tests pytest sur backend MLflow temporaire, ruff, Makefile, CI GitHub Actions (lint → tests → pipeline → build Docker)

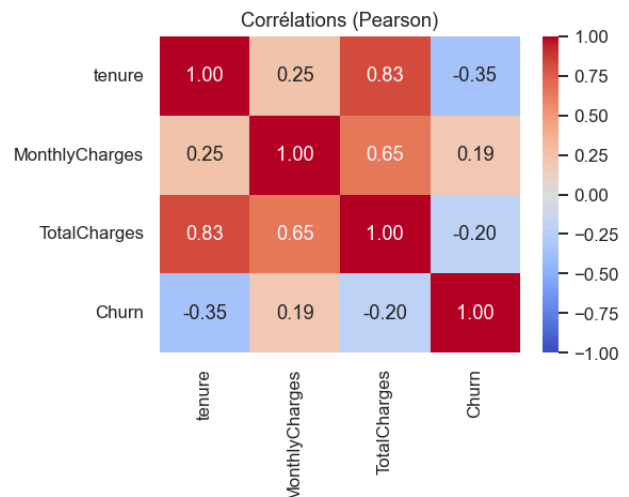
3. Exploration des données

Détail dans notebooks/eda.ipynb. Points clés :

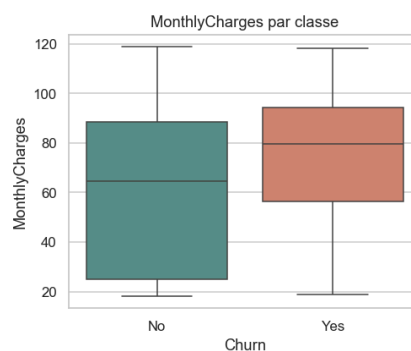
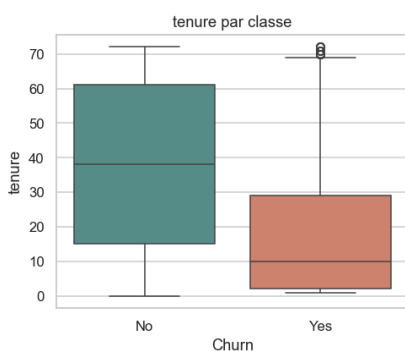
- **Qualité** : 11 valeurs TotalCharges vides, toutes pour des clients à tenure = 0 (nouveaux clients, pas encore facturés) → converties en NaN et imputées par la médiane dans le pipeline ; 22 doublons exacts supprimés ; SeniorCitizen recodée en catégorielle ; identifiant client retiré.
- **Cible** : déséquilibre modéré (26.4% de churn) → split et CV stratifiés, ROC-AUC comme métrique de sélection, class_weight dans la grille.
- **Signal** : l'ancienneté (médiane 10 mois chez les churners contre 38), le type de contrat (43 % de churn en mensuel contre 3 % en 2 ans), la fibre optique, l'absence de services de sécurité/support et le paiement par chèque électronique sont les facteurs les plus discriminants.



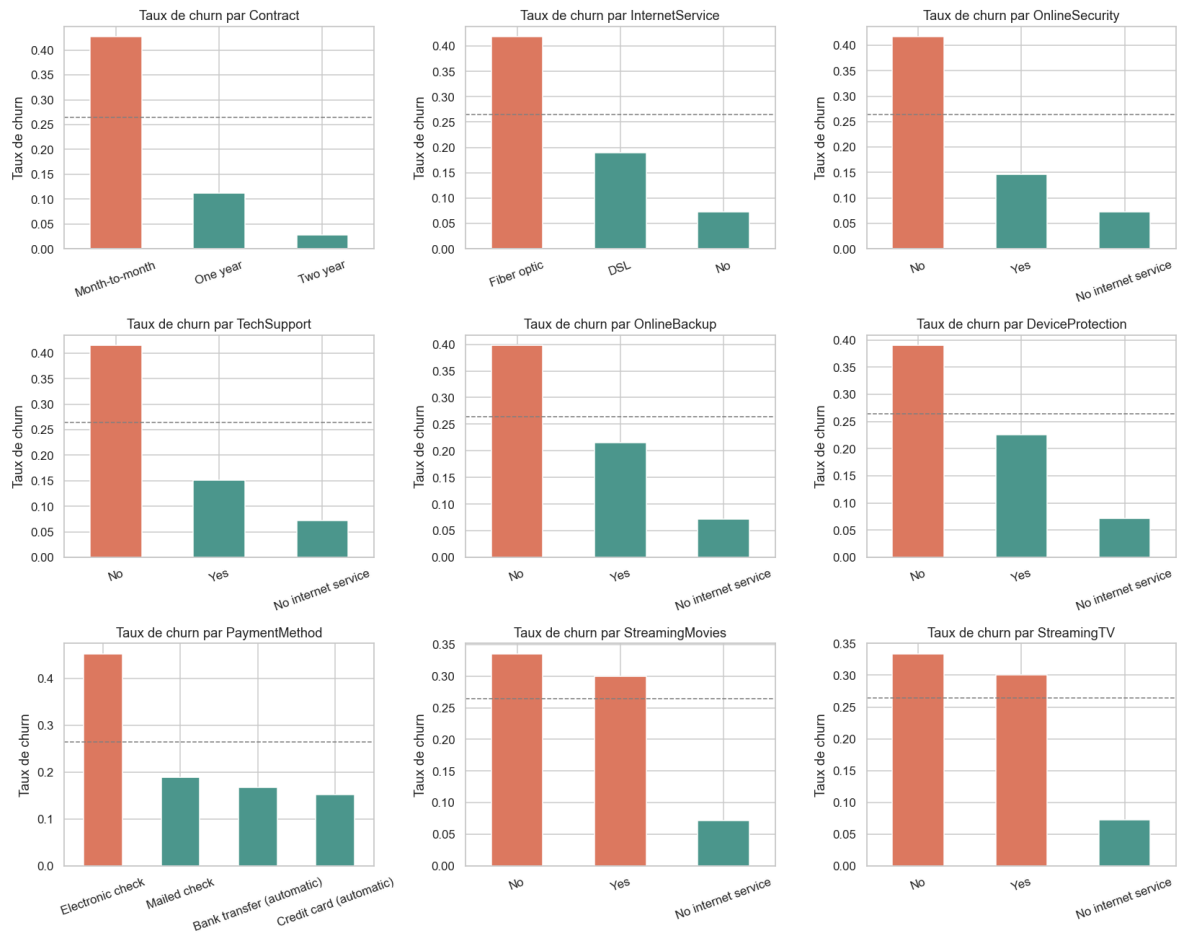
Distribution de la cible



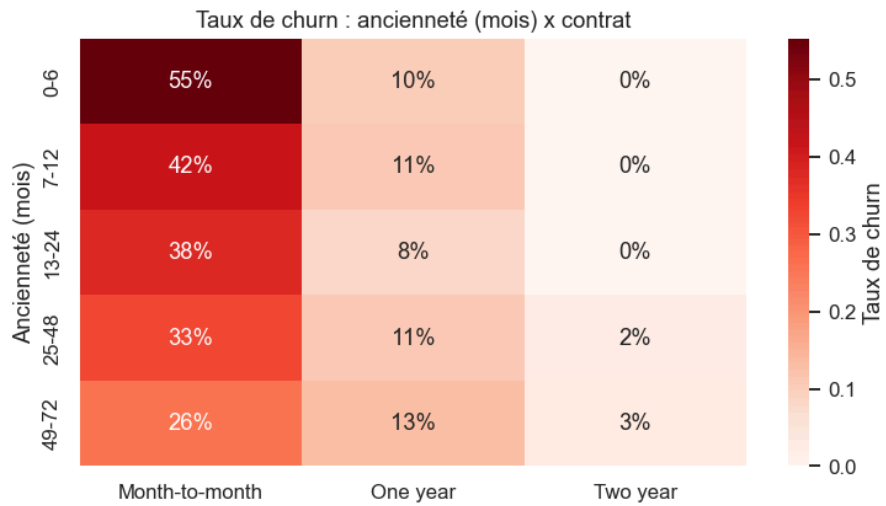
Corrélations numériques



Variables numériques par classe



Taux de churn par modalité (pointillé : taux moyen)



4. Entraînement et traçabilité MLflow

Chaque exécution de `make train` crée un run MLflow parent (un run enfant par combinaison d'hyperparamètres testée par GridSearchCV) et enregistre :

- **Paramètres** : tous les paramètres du pipeline et de la grille, plus les meilleurs (`best_model__C...`)
- **Métriques** : score CV de chaque candidat, `best_cv_roc_auc`, métriques hold-out (`test_*`), métriques d'évaluation (`eval_*`)
- **Artefacts** : `config.yaml`, `cv_results.csv`, courbes, matrice de confusion, rapport de classification, prédictions, et le **modèle** avec sa signature d'entrée/sortie, un exemple d'entrée et ses dépendances épinglées
- **Données** : empreinte (digest) et schéma du jeu d'entraînement (`mlflow.log_input`)
- **Registry** : nouvelle version de `TelcoChurnClassifier`, taguée, et alias `@champion` déplacé dessus

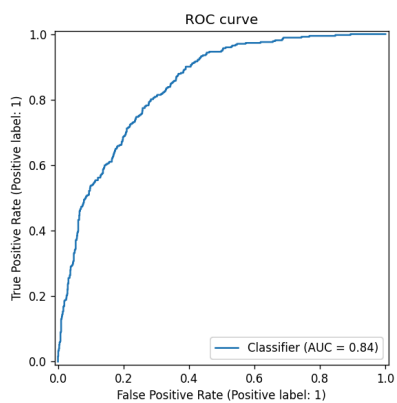
Versions enregistrées dans le Model Registry

Version	Modèle	Meilleurs hyperparamètres (CV)	CV ROC-AUC	Test ROC-AUC	Accuracy	Précision	Rappel	F1
v1	logreg	C=10.0, class_weight=None, penalty=l2, solver=liblinear	0.846	0.840	0.803	0.660	0.527	0.586
v2	random_forest	class_weight=balanced, max_depth=10, min_samples_leaf=5, n_estimators=300	0.848	0.842	0.769	0.549	0.726	0.625
v3 @champion	logreg	C=10.0, class_weight=None, penalty=l2, solver=liblinear	0.846	0.840	0.803	0.660	0.527	0.586

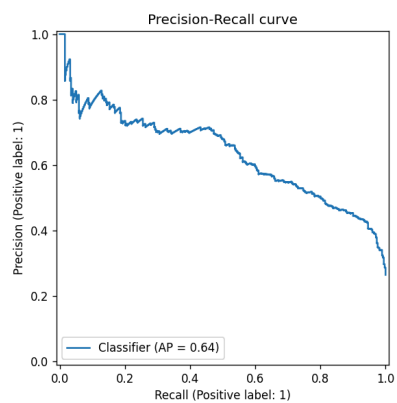
Les deux familles de modèles sont équivalentes en ROC-AUC ($\approx 0,84$). La forêt aléatoire avec `class_weight="balanced"` détecte nettement plus de churners (rappel $\approx 0,73$ contre $\approx 0,53$) au prix d'une précision plus faible : le choix du champion dépend du coût métier d'un churn manqué face à une action de rétention inutile. Le seuil de décision (0,5 par défaut) est lui aussi un paramètre de configuration.

5. Évaluation du modèle champion

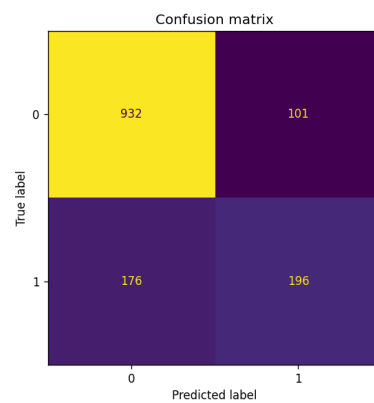
Hold-out de 20 % (stratifié, seed 42), jamais vu pendant la recherche d'hyperparamètres. ROC-AUC = **0.840**, accuracy = 0.803, précision = 0.660, rappel = 0.527, F1 = 0.586 (seuil 0.5).



Courbe ROC



Courbe précision-rappel



Matrice de confusion

6. Déploiement

L'API FastAPI (`make serve`, documentation Swagger sur `/docs`) charge `models:/TelcoChurnClassifier@champion` depuis le registry et, à défaut, l'export local `artifacts/model.joblib` embarqué dans l'image Docker. Elle expose :

Endpoint	Rôle
GET /health	liveness + source du modèle chargé
GET /model-info	version, run_id et métriques du modèle servi (traçabilité jusqu'au run MLflow)
POST /predict	un client (validation stricte des modalités et bornes) → probabilité, décision, seuil
POST /predict/batch	liste de clients

```
curl -X POST localhost:8000/predict -H "Content-Type: application/json" -d '{"tenure": 2,
"MonthlyCharges": 85.7,
"TotalCharges": 171.4, "gender": "Female", "SeniorCitizen": "0", "Partner": "No", "Dependents":
"No",
"PhoneService": "Yes", "MultipleLines": "No", "InternetService": "Fiber optic", "OnlineSecurity":
"No",
"OnlineBackup": "No", "DeviceProtection": "No", "TechSupport": "No", "StreamingTV": "Yes",
"StreamingMovies": "Yes", "Contract": "Month-to-month", "PaperlessBilling": "Yes",
"PaymentMethod": "Electronic check"}'
→ {"churn_probability": 0.77, "churn": true, "threshold": 0.5}
```

Deux modes de conteneurisation : `make docker-run` (image autonome avec healthcheck) et `make docker-up` (docker compose : serveur MLflow avec son propre stockage + API qui charge le champion depuis ce serveur). Promouvoir un nouveau modèle en production revient à déplacer l'alias dans l'UI MLflow, sans modifier le code ni reconstruire l'image.

7. Bonnes pratiques MLOps appliquées

- **Reproductibilité** : configuration unique, seeds fixées, split déterministe, versions de dépendances épinglées (un modèle sérialisé avec scikit-learn 1.7.2 est servi avec scikit-learn 1.7.2 — un écart de version a été détecté et corrigé lors du premier build Docker).
- **Traçabilité** : chaque version du registry pointe vers un run qui relie config, données (digest), hyperparamètres, métriques, figures et modèle ; l'API expose le `run_id` servi.
- **Pas de training/serving skew** : preprocessing à l'intérieur du Pipeline, mêmes coercitions de types partagées entre entraînement et API, signature MLflow.
- **Qualité continue** : tests unitaires et d'intégration (données, pipeline, tracking/registry sur backend temporaire, API), lint, CI qui rejoue le pipeline complet et construit l'image à chaque push.
- **Séparation code / données / artefacts** : `data/`, `mlruns/`, `artifacts/`, `reports/` régénérables et hors git.
- **Configuration 12-factor** : variables d'environnement prioritaires sur le YAML (`.env.example`).

8. Limites et pistes d'amélioration

- Pas de suivi de dérive (data/concept drift) en production : ajouter un job de monitoring comparant la distribution des requêtes au digest d'entraînement, et déclencher un ré-entraînement.
- Optimisation du seuil selon une fonction de coût métier plutôt que 0,5.
- Modèles supplémentaires (gradient boosting) et calibration des probabilités.
- Backend MLflow SQLite adapté au local ; en équipe, passer à PostgreSQL + stockage objet via le même docker-compose.

9. Reproduire

```
git clone https://github.com/fatimaamrch/PROJET_MLOps.git && cd PROJET_MLOps
make install && make pipeline      # data → train → evaluate, tout dans MLflow
make mlflow-ui                   # http://localhost:5000
make serve                       # http://localhost:8000/docs
make test && make lint            # 22 tests, ruff
make report                      # ce document (docs/rapport.pdf)
```